Smart Campus Placement System: Calibrated Hybrid XGBoost-kNN Placement Prediction with Company-Role Recommendation

Mopidevi Bhagyasri
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY]
Roll No. 23K91A05F9
[email address]

Pamulaparthi Harsha Vardan Reddy
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY]
Roll No. 23K91A05J0
[email address]

Mohammad Faruk Baba
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY]
Roll No. 23K91A05F3
[email address]

Prahalladh Terati
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY]
Roll No. 23K91A05J8
[email address]

S Usha Devi
[DESIGNATION], Project Guide
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY]
[email address]

***Abstract*---**Campus placement cells keep student records but rarely use them to tell a student, early and with a probability, whether a target company is within reach. This paper presents the Smart Campus Placement System, which estimates the probability that a student is placed at a chosen company and role, shows the gaps against that company's cutoffs, and recommends company-role alternatives. It adapts a hybrid framework from graduate-admission prediction: a Platt-calibrated XGBoost model whose borderline cases are refined by a nearest-neighbour model, served through a FastAPI back end and a React interface. On a synthetic dataset of 3,000 students and 30 companies, calibrated XGBoost reaches 0.8067 accuracy on 450 held-out students and logistic regression has the highest AUC (0.9106). The hybrid reaches 0.8400 only when the refined cases are chosen using the true test labels; with a rule that can run at serving time it scores 0.8089. Recommendations raise the estimated probability for most weak profiles but take three to eight seconds per request. All data are synthetic, so the figures describe software behaviour, not real placement outcomes.

***Index Terms*---**calibration, campus placement, educational data mining, gradient boosting, hybrid model, placement prediction, recommendation.

I. INTRODUCTION

Campus recruitment shapes the careers of students and the reputation of the institution, and placement cells manage it with spreadsheets, notices and manual shortlisting. Student records already hold predictive signals - academic scores, aptitude and technical tests, internships, backlogs - and machine learning can turn them into a placement probability before the season starts [1]. Recent work on graduate admissions adds two ideas that transfer to placement: probabilities that are calibrated, and a local model that revisits the borderline cases a global model finds hard [2].

Today, a student usually learns whether a company was within reach only after applying, with no estimate beforehand and no statement of which gap - grade, aptitude or technical skill - decided the outcome. A recruiter filters a long list by hand, and the placement officer cannot see readiness across branches or which training would help most. The cost is late preparation, wasted applications and uneven shortlisting.

A placement officer therefore needs three answers that a stored table cannot give: which students are close to ready for which kind of company, what separates a borderline student from a ready one, and where a student who is unlikely to be placed at a dream company has a realistic alternative. A student needs the same answers in personal form. Because wrong advice has a cost - a student who aims too low loses an opportunity and one who aims too high loses a season - the estimate must be a probability that can be checked, not only a yes or no.

The studies surveyed in Section II mostly report a classification accuracy on a small table of student records. Few give a calibrated probability, name the gap that holds a student back, or recommend alternative companies, and fewer still ship a working application. The declared base paper reports no numeric result at all, and the technique paper solves admissions, not placement. This work fills that gap and, in addition, measures how much of the hybrid model's advantage survives when it is evaluated without peeking at the answers.

The contributions are: (1) a working placement-prediction system that returns a calibrated probability and rule-based skill-gap tips for a chosen company and role; (2) a three-strategy company-role recommendation module with feasibility filters; (3) an officer dashboard and recruiter shortlist behind a FastAPI service and a React interface over unchanged model code; and (4) an evaluation across nine models, five random splits, a threshold and band analysis, calibration, a label-free hybrid rule, 46 automated checks and latency, stated together with what was not measured. Section II reviews related work, Section III gives the method, Section IV the results and Section V concludes.

A. System Hardware

All training and measurement ran on one desktop: Intel Core i5-12500 (6 cores, 12 threads), 15.7 GB RAM, none (CPU only), Windows 11 Pro. Retraining one full split (six baselines, calibration and two local models) takes about three seconds.

B. Software Requirements

Python 3.14.7, scikit-learn 1.9.0 [3], XGBoost 3.4.1 [4], SHAP 0.52.0 [5], pandas 3.0.5, NumPy 2.5.3, FastAPI 0.159.1 with Pydantic 2.14.1, Flask 3.1.3 for the original interface, and React 19.3 with Vite 8.3 on Node 24.19.0 for the browser interface.

II. LITERATURE SURVEY

Declared base paper. Prasanna Kumari et al. [1] describe a Streamlit placement-prediction application: a synthetic dataset of student records, Logistic Regression, Random Forest and Gradient Boosting trained on an 80:20 split, scaling and tuning, and extras such as batch CSV prediction, skill-gap analysis, a PDF report and e-mail delivery. The paper states the metrics it would use - accuracy, precision, recall, F1 and ROC-AUC - but reports no value for any of them, gives no dataset size, and offers no company recommendation. It is therefore a design reference, not a numeric benchmark.

Technique base. Heidari Fard and Tabrizi [2] predict graduate admission on 13,000 real GradCafe records enriched with QS rankings. XGBoost [4] is calibrated by Platt scaling [6] and its residual cases - those it gets wrong or scores between 0.4 and 0.6 - are re-predicted by a k-nearest-neighbour model [7]; accuracy rises from 0.7241 to 0.8723 and Brier score [8] is 0.1864. A dual-axis recommendation module with feasibility filters raises the mean acceptance probability of rejected applicants by 65.1 %. This project reuses that framework, replacing universities by companies, programs by roles and QS rank by company tier.

Placement prediction. Thangavel et al. [9] classify students into placement statuses with decision trees; Khandale and Bhoite [10] compare eight classifier families and report about 78 % accuracy for XGBoost and AdaBoost. Baffa et al. [11] reach 98 % with a random forest [12] on 218 graduates of one department; Ambili and Abraham [13] report 90.21 % for a stacking ensemble; Gupta et al. [14] 0.9524 accuracy; Pathak et al. [15] 0.8923 accuracy and 0.9433 AUC for a voting classifier on a public Kaggle table; and Sanjana et al. [16] pair a support-vector classifier [17] with a deep Q-network for scheduling and job matching. Surveys of educational data mining [18] list placement and performance prediction among its standard tasks.

TABLE I
REPRESENTATIVE WORK COMPARED

Work	Approach	Data	Reported result	Limitation
[1]	LR, RF, gradient boosting; Streamlit app	Synthetic, size not stated	No numeric result	No evaluation; no recommendation
[2]	XGBoost + Platt; kNN/SVM residual; recommendation	13,000 GradCafe records	Acc. 0.7241 to 0.8723	Residual set uses labels; admissions
[9]	Decision trees, five statuses	Historical records	Abstract only	No probability
[10]	Eight classifier families	Past placed/unplaced	About 78 %	Moderate accuracy
[11]	LR, DT, RF on six variables	218 graduates, one university	RF 98 %, F1 0.99	Tiny single-department set
[13]	Stacking, 20 attributes	Size not stated	90.21 %	No calibration
[14]	Classifiers + ensemble	Recruitment data	Acc. 0.9524, F1 0.9286	Single split
[15]	Tuned voting classifier	Public Kaggle set	Acc. 0.8923, AUC 0.9433	No recommendation
[16]	SVM + deep Q-network	Not described	87.4 % (SVM)	Data undescribed

Methods used by this line of work. Gradient-boosted trees such as XGBoost [4] and random forests [12] dominate tabular prediction because they capture thresholds and interactions, such as a CGPA cutoff that depends on company tier, without hand-built features. Their raw scores are not probabilities, which matters when a student is told a percentage and when candidates are ranked; Platt scaling [6] and the comparison of calibration methods by Niculescu-Mizil and Caruana [19] address this. Nearest-neighbour rules [7] give a local view of the training data, which is the intuition behind the residual refinement. The studies above mostly stop at accuracy; they rarely report calibration, ranking quality, variation over random splits or a cost for the answer that a deployed service gives.

Recommendation and explanation. Recommendation applications [20] rank candidates by predicted suitability; SHAP [5] attributes a model's output to features, and calibration [19] makes probabilities usable for ranking. Results in Table I come from different data and metrics and are not ranked: several use small or undescribed tables, and none combines a calibrated probability, skill-gap feedback and company recommendation in one application. That combination, evaluated honestly on the data available, is the gap this paper addresses.

III. METHODOLOGY

Data. No real placement records were available, so 3,000 students with 19 columns and a table of 30 companies (7 Tier-1, 15 Tier-2, 8 Tier-3, with CGPA, aptitude and technical cutoffs, package, location and hiring rate) were generated with seed 42. Scores derive from CGPA plus noise; the label is drawn from a logistic score of the gaps between the student and the target company's cutoffs, a skill index, internships, projects, certifications, backlogs, a bonus for CSE and IT, a penalty by tier and Gaussian noise. Of the students, 59.7% are placed; the placed rate falls from 81.6% for Tier-3 targets to 9.6% for Tier-1. Because the label follows a known rule, models recover the rule, and every number below describes software, not campuses.

Features and split. Five categorical columns are frequency-encoded and 13 numeric plus six engineered columns are standardised, giving 24 features. The engineered columns are the skill index (0.4 technical + 0.3 aptitude + 0.15 communication + 0.15 DSA, over 100), the company hiring rate, CGPA times tier, and the three gaps CGPA, aptitude and technical score minus the company cutoff. Encoders and scalers are fitted on the training part only. A stratified 70/15/15 split gives 2,100 training, 450 validation and 450 test students.

Skill-gap tips. Tips are rules, not model output: for the target company the system compares CGPA, aptitude and technical score with its cutoffs and names each one that falls short, adds a prompt for an internship when there is none,

and otherwise states that the profile is strong. They are deliberately transparent so that a student can act on them; the global SHAP ranking shown to officers explains the model, not the individual.

Models. Six baselines are trained: logistic regression, a decision tree, a random forest [12] of 200 trees, an RBF support-vector machine [17], k-nearest neighbours with k = 9, and XGBoost with 100 trees, depth 8, learning rate 0.1, subsample and column sampling 0.8, L1 0.1 and L2 1. XGBoost is calibrated with Platt scaling [6] (sigmoid, three folds),

$$P(y = 1 \mid f) = 1 / (1 + \exp(A f + B)), \qquad (1)$$

and a 9-fold cross-validation of the uncalibrated model on the training part serves as a stability check. Hyper-parameters are those of the technique paper and were not tuned further, so the baselines are not individually optimised; this keeps the comparison simple but means a tuned model could score differently.

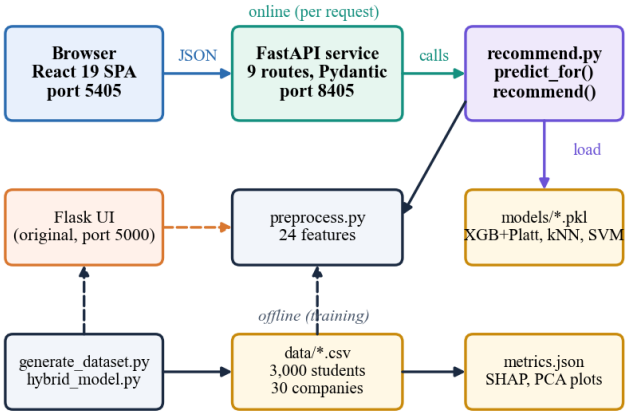


Fig. 1. System architecture: browser, FastAPI service, shared prediction modules and file storage; training runs offline.

Hybrid residual rule. Following the technique paper, the residual set is

$$R = \{\, i : \hat{y}_i \neq y_i \;\; or \;\; 0.4 \le p_i \le 0.6 \,\}, \qquad (2)$$

and predictions in R are replaced by those of a kNN (or SVM) fitted on all training rows. The first condition needs the true label of the student, so it cannot be evaluated for a new student. Two label-free variants are therefore also measured: replacing only the borderline rows by the kNN label, and the serving rule that the application actually uses for a borderline probability p,

$$p' = 0.5\, p + 0.5\, p_{kNN}, \quad 0.4 \le p \le 0.6. \qquad (3)$$

A student is labelled placement ready when the final probability reaches 0.5, and rule-based tips name each of CGPA, aptitude and technical score that is below the company cutoff and a missing internship.

Recommendation. For a profile, each of the 30 companies is kept only if |CGPA - cutoff| is within Δ_max (0.6, 1.0 or 1.5 for tiers 1, 2 and 3), the aptitude is at most 15 points below the cutoff, and the student has at most two backlogs for a Tier-1 company. For each kept company four roles are scored with

$$s = p + 0.05\, prox + 0.02\, loc, \qquad (4)$$

$$prox = \max(0,\, 1 - |CGPA - c| \,/\, \Delta_{max}), \qquad (5)$$

and three lists are returned: the best role per company, the target company with other roles, and the best five company-role pairs overall. The reported gain is the mean of p minus the probability for the original target over the pairs that beat it (Fig. 2).

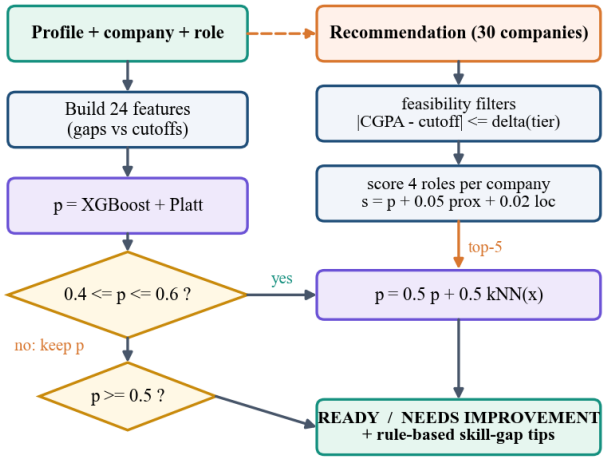


Fig. 2. Main flow: probability with borderline refinement (left) and company-role recommendation (right).

Evaluation protocol. Accuracy, precision, recall, F1 and ROC-AUC are computed on the test part at the 0.5 threshold; the Brier score is the mean squared difference between the predicted probability and the outcome. With precision P and recall R,

$$F1 = 2PR / (P + R). \qquad (6)$$

Because one split of 450 students gives noisy differences, the whole procedure - split, encoders, six baselines, calibration and both hybrids - is repeated for five seeds and means and standard deviations are reported. The shipped model files (seed 42) are used for the confusion matrices, band and threshold sweeps, SHAP ranking and recommendation study, so that they match the application's own dashboard.

Implementation. Fig. 1 shows the layers. A FastAPI service exposes nine routes: health, metadata, companies, metrics, dataset statistics, three explanation plots, prediction, recommendation and recruiter shortlist. Pydantic limits every field (CGPA 0-10, scores 0-100, counts 0-20), an unknown company returns 404 and the browser origin is the only one allowed. The service imports the original prediction functions unchanged; each request reloads four pickle files. The React interface has pages for eligibility, recommendations, an officer dashboard (metrics, SHAP, PCA, probability histogram, branch rates), a recruiter shortlist and a company board, with dark and light themes, a command palette and an activity log that lists every call the interface makes. Forty-six automated checks (unit, integration, functional, validation, security) and a 15-check smoke test pass.

Design decisions. The FastAPI layer imports the existing prediction code instead of rewriting it, so the original Flask interface and the new one give the same numbers. Pickle files

are reloaded on every request, which keeps the code simple and the models always current but costs seconds for recommendations. Frequency encoding avoids a sparse one-hot expansion for company and role names; the gap features give the model the quantity that matters, the distance to a company's cutoffs. A single 0.5 threshold is used because accuracy is flat across thresholds (Section IV), and CSV and pickle files replace a database because the dataset is small and read-only.

IV. RESULTS

Test conditions. Models are scored on the 450 held-out students of the stratified split with seed 42 (59.6 % placed). The shipped model files reproduce the stored metrics exactly. Differences between models are small against the sampling noise of 450 students: one standard deviation of accuracy over five random splits is 0.01-0.02. Table II lists the single-split metrics. The data are synthetic, so no claim is made about real campuses.

TABLE II
TEST-SET METRICS, SEED 42 (450 STUDENTS)

Model	Acc.	AUC	F1	Prec.	Rec.
Logistic Reg.	0.8289	0.9106	0.8577	0.8498	0.8657
Decision Tree	0.7533	0.7506	0.7869	0.8103	0.7649
Random Forest	0.8067	0.8918	0.8404	0.8267	0.8545
SVM (RBF)	0.8289	0.8855	0.8597	0.8399	0.8806
kNN (k = 9)	0.7978	0.8638	0.8378	0.8020	0.8769
XGBoost	0.8089	0.8888	0.8396	0.8396	0.8396
XGBoost + Platt	0.8067	0.8928	0.8404	0.8267	0.8545
Hybrid, labels used	0.8400	0.8968	0.8710	0.8379	0.9067
Hybrid, label-free	0.8089	0.8954	0.8442	0.8204	0.8694
Hybrid, serving rule	0.8089	0.8939	0.8436	0.8227	0.8657

Key finding. The hybrid with the technique paper's residual rule scores 0.8400 against 0.8067 for calibrated XGBoost (+3.3 points, recall 0.8545 to 0.9067). The gain comes from the selection: XGBoost is wrong on 87 of the 450 test students, 72 of them outside the borderline band, and they enter the 103-student residual set only because their true label shows they are wrong. On that set the kNN is right for 30.1% against 15.5% for XGBoost - a selection effect. When only the 31 borderline students are re-predicted, which is possible for a new student, accuracy is 0.8089 (serving rule 0.8089), one student more than XGBoost. The kNN is right on 54.8% of the borderline rows against 51.6%. The technique paper's 0.8723 uses the same label-dependent rule, so it should be read with the same caution.

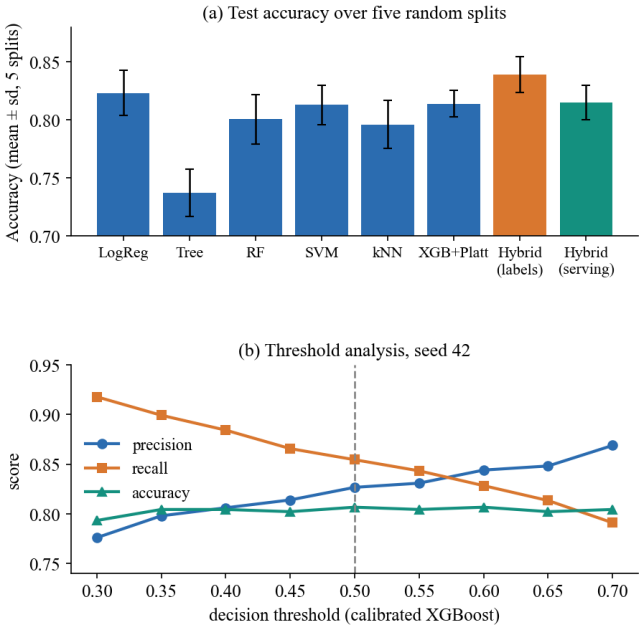


Fig. 3. Accuracy over five random splits (a) and threshold analysis of the calibrated model (b).

Five splits. Over seeds 42, 1, 2, 3 and 4 the mean accuracy is 0.8391 ± 0.0154 for the labels-used hybrid, 0.8142 ± 0.0180 label-free, 0.8151 ± 0.0149 for the serving rule, 0.8138 ± 0.0115 for XGBoost with Platt scaling and 0.8231 ± 0.0194 for logistic regression; the latter has the highest AUC (0.9031 ± 0.0109) because the synthetic label is a logistic function. The labels-used hybrid is ahead of XGBoost by 2.5 points on average and the label-free versions by 0.0-0.1 points, well inside one standard deviation (Fig. 3a).

TABLE III
FIVE-SPLIT ACCURACY AND TEST CONFUSION COUNTS

Model	Acc. (mean ± sd)	TN	FP	FN	TP
Logistic Reg.	0.8231 ± 0.0194	141	41	36	232
XGBoost + Platt	0.8138 ± 0.0115	134	48	39	229
Hybrid, labels used	0.8391 ± 0.0154	135	47	25	243
Hybrid, serving	0.8151 ± 0.0149	132	50	36	232

TABLE IV
REPLACING BORDERLINE PREDICTIONS BY KNN (SEED 42)

Band of p	Replaced	Acc.	F1	AUC
none	0	0.8067	0.8404	0.8928
0.45-0.55	13	0.8022	0.8379	0.8935
0.40-0.60	31	0.8089	0.8442	0.8954
0.35-0.65	45	0.8067	0.8444	0.8967
0.30-0.70	73	0.8067	0.8455	0.8941
0.20-0.80	133	0.8067	0.8449	0.8886

Band, threshold and calibration. Table IV shows that widening the band from none to 0.2-0.8 replaces up to 133 students and keeps accuracy between 0.8022 and 0.8089: the local model adds no measurable value on these data. Moving the decision threshold from 0.30 to 0.70 changes accuracy by under 1.5 points (0.7933-0.8067) but trades precision (0.776 to 0.869) against recall (0.918 to 0.791), so 0.5 is a reasonable middle (Fig. 3b). Platt scaling lowers the Brier score [8] only from 0.1379 to 0.1352; logistic regression is

better (0.1180) and the mean predicted probability (0.592) matches the observed rate (0.596).

Patterns in the data. The placed rate depends strongly on the target tier (9.6%, 49.5% and 81.6% for tiers 1-3), falls with backlogs from 72.2% with none to 33.7% with four, and rises with internships from 56.8% to 78.6%; branches differ by under three points (58.5%-60.8%). These patterns are built into the generator, which is why the models find them.

Explanation. Mean absolute SHAP value [5] ranks the CGPA gap (1.12), backlogs (0.88), the technical gap (0.50), the aptitude gap (0.40) and CGPA times tier (0.39) first, well above all other features (below 0.18). This mirrors how the labels were generated, so it validates the explanation tool, not a fact about real placements (Fig. 4). Fig. 5 shows the eligibility screen that presents one such prediction with its cutoff checks.

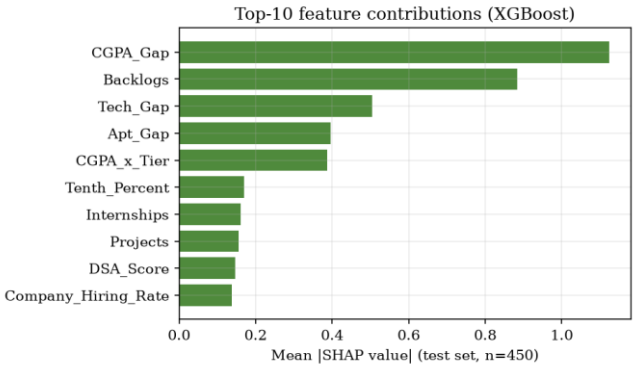


Fig. 4. Mean absolute SHAP value of the ten most influential features for XGBoost on the test students.

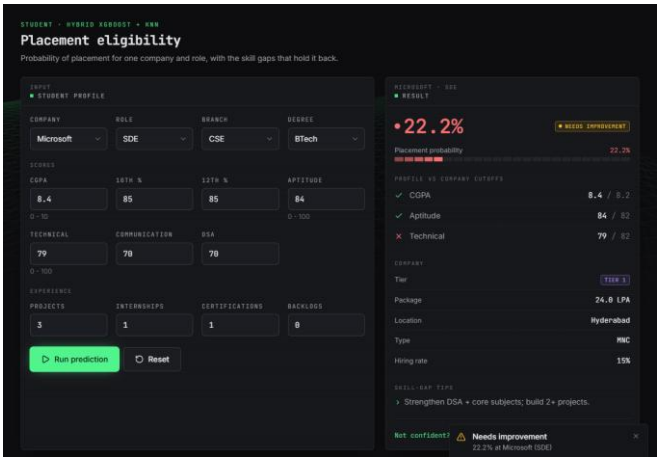


Fig. 5. Eligibility screen: Microsoft, SDE, CGPA 8.4, aptitude 84, technical 79 gives 22.2 %, with cutoff checks and a tip.

Recommendation. For 60 test students rated below 0.5 for their own target, the mean probability is 0.222; 52 of 60 receive at least one better pair, the top pair averages 0.643, and it reaches 0.5 for 39 of 60 (65%). These are model estimates, not realised placements. In Fig. 6 a profile aimed at Google (10.8 %) is steered to ValueLabs and Infosys, Tier-3 companies, because the score maximises probability, not package or fit; and when the target fails the CGPA filter the role-only list is empty.

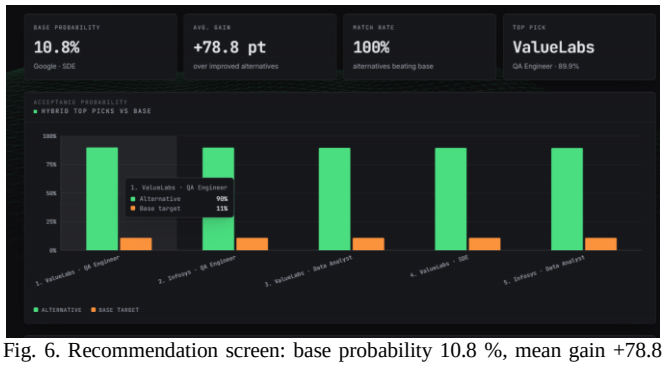


Fig. 6. Recommendation screen: base probability 10.8 %, mean gain +78.8 points, top pick ValueLabs QA Engineer 89.9 %.

Speed and tests. On localhost (127.0.0.1) a prediction takes 66 ms on average (110 ms at the 95th percentile) and a shortlist 28 ms, but a recommendation takes 5.4 s on average, and 3.5 s in process, because every scored pair reloads the four model files; caching them is the obvious fix and was not done. All 46 automated checks pass, including malformed input, injection-style strings and path traversal; an unknown branch or role is silently accepted, and there is no authentication.

Failure cases. Calibrated XGBoost makes 48 false placements and 39 missed placements among 450 students; moving the threshold shifts errors between the two kinds but not their total. The model has not seen real borderline students, so its probability near 0.5 should be read as an ordering, not a promise. Further observed limits: the recommendation module sends a weak profile to the easiest companies, the role-only list is empty when the target fails the CGPA filter, unseen categories get a frequency of zero instead of an error, and tips cannot say how large a gap is worth closing.

Test coverage. The 46 checks are 10 unit, 8 integration, 6 functional, 13 validation and 9 security checks. They confirm that probabilities stay between 0 and 1, rise with CGPA and fall with backlogs, that an all-zero profile scores 0.139 and an all-maximum profile 0.799 at Google, that out-of-range values return 422, and that a 100 kB company name, a script tag and a path-traversal plot name are all rejected with 404. They do not include a browser-automation suite, a load test or any real student data.

Comparison with the base papers. The declared base paper reports no numbers, so no comparison is possible. The technique paper's 0.7241 (XGBoost) and 0.8723 (hybrid) are on 13,000 real, noisy admission records; our 0.8067 and 0.8400 are on a synthetic table generated by a logistic rule, on which even logistic regression reaches 0.8289. The two sets differ in size, origin, noise and task, so the numbers are not comparable; the relative result that carries over is that the hybrid's advantage depends on how the residual set is chosen.

V. CONCLUSION

The system delivers what the batch objectives asked for in software: a calibrated placement probability with skill-gap tips, a three-strategy company-role recommendation, an officer dashboard and a recruiter shortlist, all behind a tested API. Six classifiers, a calibrated XGBoost and two hybrids

were compared over five splits. The central finding is negative for the hybrid: its 0.84 accuracy depends on selecting residual cases with the true labels, and with a rule usable for a new student it equals calibrated XGBoost (0.8089 against 0.8067); logistic regression ties or beats every model on AUC.

Against the six objectives: data were generated, not collected; encoding was done and the 59.7/40.3 class ratio needed no resampling; classifiers were trained and compared; recommendation, skill-gap tips and dashboards were built; and accuracy, precision, recall, F1, AUC and Brier score were reported. The experiment also shows that a simple linear model is a strong baseline when the signal is linear, and that any claimed gain from refining borderline cases must be checked without access to the answers.

The limits are plain: the data and company table are synthetic, so no real-campus accuracy is claimed; recommendation gains are unverified estimates; recommendations take seconds; tips are fixed rules; and there is no database, login, e-mail or PDF report. Future work: train on real placement records from the cell, cache the model files, train the refinement without label leakage, add per-student SHAP, authentication with roles and a database, batch CSV prediction and a fairness audit across branches.

REFERENCES

[1] M. Prasanna Kumari, B. Punith, M. Srinath, P. Koushik and Y. Gourav, "Student placement prediction system using machine learning," Int. J. Eng. Technol. Manage. Sci. (IJETMS), vol. 10, no. 3, pp. 47-51, May-Jun. 2026, doi: 10.46647/ijetms.2026.v10i03.008.

[2] M. Heidari Fard and E. Tabrizi, "A hybrid machine learning approach for graduate admission prediction and combined university-program recommendation," arXiv:2603.29881 [cs.IR], Feb. 2026.

[3] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, Oct. 2011.

[4] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining, San Francisco, CA, USA, Aug. 2016, pp. 785-794.

[5] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems 30 (NIPS 2017), Long Beach, CA, USA, Dec. 2017, pp. 4765-4774.

[6] J. C. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in Advances in Large Margin Classifiers, A. J. Smola, P. Bartlett, B. Scholkopf and D. Schuurmans, Eds. Cambridge, MA, USA: MIT Press, 1999, pp. 61-74.

[7] T. Cover and P. Hart, "Nearest neighbor pattern classification," IEEE Trans. Inf. Theory, vol. 13, no. 1, pp. 21-27, Jan. 1967.

[8] G. W. Brier, "Verification of forecasts expressed in terms of probability," Mon. Weather Rev., vol. 78, no. 1, pp. 1-3, Jan. 1950.

[9] S. K. Thangavel, P. D. Bkaratki and A. Sankar, "Student placement analyzer: A recommendation system using machine learning," in Proc. 4th Int. Conf. Adv. Comput. Commun. Syst. (ICACCS), IEEE, 2017 (authors as listed in the Amrita University publication record).

[10] S. Khandale and S. Bhoite, "Campus placement analyzer: Using supervised machine learning algorithms," Int. J. Comput. Appl. Technol. Res. (IJCATR), vol. 8, no. 9, pp. 358-362, 2019.

[11] M. H. Baffa, M. A. Miyim and A. S. Dauda, "Machine learning for predicting students' employability," UMYU Scientifica, vol. 2, no. 1, pp. 1-9, Mar. 2023, doi: 10.56919/usci.2123_001.

[12] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5-32, Oct. 2001.

[13] P. S. Ambili and B. Abraham, "A comprehensive evaluation of employability prediction using ensemble learning techniques," EPRA Int. J. Multidiscip. Res. (IJMR), vol. 10, no. 1, Jan. 2024.

[14] A. Gupta, S. Gupta, P. K. Mall, S. Srivastava, A. S. Saluja, N. Yadav, V. Narayan, M. K. Verma and S. Sriramulu, "ML-CPC: A pathway for machine learning based campus placement classification," J. Electr. Syst., vol. 20, no. 3s, 2024, doi: 10.52783/jes.1521.

[15] A. Pathak, M. Matcha, M. Gopisetti and S. Joshi, "A machine learning framework for predicting student placement outcomes," Ing. Syst. Inf. (ISI), vol. 30, no. 7, pp. 1715-1721, Jul. 2025, doi: 10.18280/isi.300704.

[16] N. V. S. Sanjana, M. Varun, M. Greeshma, S. Ponugupati, M. Gurupriya and G. Ramaswamy, "Revolutionizing university placements: Advanced technologies for streamlined and student-centric ecosystems," in Proc. 1st Int. Conf. Res. Develop. Inf. Commun. Comput. Technol. (ICRDICCT), vol. 2, SCITEPRESS, 2025, pp. 83-88, doi: 10.5220/0013877200004919.

[17] C. Cortes and V. Vapnik, "Support-vector networks," Mach. Learn., vol. 20, no. 3, pp. 273-297, Sep. 1995.

[18] C. Romero and S. Ventura, "Educational data mining: A review of the state of the art," IEEE Trans. Syst., Man, Cybern. C, Appl. Rev., vol. 40, no. 6, pp. 601-618, Nov. 2010.

[19] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in Proc. 22nd Int. Conf. Mach. Learn. (ICML), Bonn, Germany, Aug. 2005, pp. 625-632.

[20] J. B. Schafer, J. A. Konstan and J. Riedl, "E-commerce recommendation applications," Data Min. Knowl. Discov., vol. 5, no. 1-2, pp. 115-153, Jan. 2001, doi: 10.1023/A:1009804230409.